实验二 彩票调度

目标

- 熟悉调度程序的工作原理。
- 将调度程序更改为新的彩票调度算法。

准备工作

- 虚拟机内下载实验代码

```
git clone https://github.com/mit-pdos/xv6-public.git
```

注：若下载超时，可以在**虚拟机**的Firefox浏览器输入网址https://github.com/mit-pdos/xv6-public.git，并下载代码压缩包

实验要求

- 彩票调度的**基本思想**：根据进程持有的彩票数量的比例，为每个运行的进程分配CPU。每个时间片，随机抽签决定彩票的赢家;赢家是在那个时间片上运行的过程。进程拥有的彩票越多，运行的次数就越多。
- **填充pstat结构：** 在内核中填充`pstat`结构，存储每个进程的彩票信息的数据结构，并将结果传递到用户空间。`pstat`结构不能进行更改，必须按照下面给定的结构使用。

  ```
  struct pstat {
    int inuse[NPROC];   // 是否可运行（1或0）
    int tickets[NPROC]; // 此进程拥有的彩票数
    int pid[NPROC];     // 每个进程的进程ID
    int ticks[NPROC];   // 每个进程已经运行的CPU时间片数量（被选择运行的次数）
  };
  //NPROC:最大进程数
  ```

- **新增2个系统调用：**
  - `int settickets(int number)`：设置调用进程的彩票数量。默认情况下，每个进程被分配5个彩票。调用`settickets()`使得进程可以增加彩票数量，从而获得更高比例的CPU周期。如果成功，返回0；否则返回-1（例如，如果调用者传入一个小于1的数字）。
  - `int getpinfo(struct pstat *)`：返回有关所有运行进程的信息，包括`struct pstat`的4个成员变量信息。可以用此系统调用构建`struct pstat *ps`。返回0表示成功，返回-1表示失败(例如，如果向内核传递了一个坏指针或NULL指针)。
- **进程继承彩票：** 在创建进程时需要为其分配彩票数。具体来说，确保子进程在创建时继承父进程的彩票数量。如果父进程有10张彩票并调用`fork()`创建子进程，子进程也应该获得10张彩票。
- **生成随机数：** 在内核中实现伪随机数生成器，以用于彩票调度的随机彩票抽奖。
- **最终结果**：打印每个进程的信息，包括：进程id（pid），是否可运行（1或0），彩票数（tickets），划分的时间片数量（ticks），以及总时间片数（total ticks）。

实验说明

1. **修改与创建文件**：

- 修改 syscall.h、syscall.c、defs.h、proc.h、param.h、proc.c、sysproc.c、usys.S 、user.h 、Makefile文件，以支持新的彩票调度器。
- 创建新的文件：pstat.h、rand.c、rand.h、testTicket.c、testProcInfo.c

2. **实现用户空间的testTicket.c、testProcInfo.c：**
   - testTicket.c：设置当前进程的票数。它接受一个整数参数作为命令行参数，并将其转换为票数。然后它使用 `settickets()` 系统调用来设置当前进程的票数为给定的值。主要作用是在用户空间调整进程的调度优先级。
   - testProcInfo.c：显示当前正在运行的进程的信息，包括进程ID、是否可运行、分配的彩票数和已累积的CPU时间片数（ticks）。它通过调用 `getpinfo()` 系统调用来获取进程信息，然后将其打印到标准输出。主要作用是提供了一个简单的方式来查看系统中各个进程的调度情况和资源利用情况。
3. **引入伪随机数生成器**：
   - `rand.c`、`rand.h`的实现，将伪随机数生成器添加到了xv6 内核中。
   - 在 `proc.c` 中实现 `scheduler()` 函数时调用rand.c中的`random_at_most(total)`即可，传入参数total应为彩票总数。
4. **实现 settickets 系统调用**：
   - 在 `proc.h` 中修改 `struct proc`，为每个进程添加 `ticket`（跟踪每个进程持有的彩票数量）、`tick`（当前运行的CPU时间片）字段。
   - 在`proc.c`和`proc.h`中，实现 `settickets` 系统调用，用于为调用进程设置票数。`sysproc.c`的`sys_settickets()`中通过 `argint` 函数从用户空间获取参数。
5. **实现 getpinfo 系统调用**：
   - 在`proc.c`和`proc.h`中，实现 `getpinfo` 系统调用，返回所有运行中进程的信息。`sysproc.c`的`sys_getpinfo()`中使用 `argptr` 获取用户空间传递的结构体指针。
6. **在创建和初始化进程时添加彩票信息**：
   - 修改`proc.c`的`allocproc()`、`fork()`函数
7. **调整调度算法**：
   - 修改 `proc.c` 中 `scheduler()` 函数，实现彩票调度。需要将原有的轮询调度算法修改为彩票调度。
   - 需要用到`rand.h`中的`random_at_most()`函数，生成伪随机数
8. **测试**：

```
$make qemu-nox    (运行qemu模拟器)

$testTicket 10&; testTicket 15&; testTicket 20&; testTicket 25&; testTicket
30&;        (设置进程的彩票数)

$testProcInfo     (输出彩票调度后的进程状态)
```

   - 输出结果参考：

```
------------ Initially assigned Tickets = 5 ----------

ProcessID   Runnable(0/1)   Tickets       Ticks
1              1                5           2
2              1                5           2
16             1                5           0
7              1                10          72
9              1                15          127
11             1                20          151
13             1                25          185
15             1                30          229

------------------- Total Ticks = 768 -----------------
```

注意事项

- 需要对进程在创建时分配彩票进行处理，确保子进程继承父进程的彩票数量。
- 根据输出结果调整算法，确保高票数的进程被正确选择。
- 总结输出结果，确保票数多的进程执行时间更长。
- 对全局彩票总数的操作应该在保证数据一致性的前提下进行，需要注意加锁的问题。
- 注意内核对于从用户空间传递的指针非常谨慎，必须仔细检查以防止安全威胁。